

De Novo Chloroplast Assembly

The code I'm providing has absolute pathways to our data server on Sol, ASU's supercomputer. To replicate the analysis, you will need to update those pathways as well as any Mamba environments referenced within the shell scripts.

Step 1: QC Fastq Files

Run fastqc and multiqc quality control on the raw fastq files to evaluate read depth and quality before beginning the analysis.

```
module load fastqc-0.12.1-gcc-11.2.0
cd /data/gencore/analysis_projects/8710018_Sunidhi/fastq
fastqc -t 2 *

mkdir -p /data/gencore/analysis_projects/8710018_Sunidhi/qc
cd /data/gencore/analysis_projects/8710018_Sunidhi/qc
mv /data/gencore/analysis_projects/8710018_Sunidhi/fastq/*fastqc* ./

module load mamba/latest
source activate /data/biocode/programs/mamba-envs/multiqc.v1.20/
multiqc *
```

Step 1: Trim Reads

The initial step is to remove any adapter sequence from the raw reads. The assembler I'm planning to use doesn't recommend running quality control, however. They recommend >5G per end but I'm not sure if that is accounting for any remaining nuclear reads. Hopefully, since this was a chloroplast extraction specifically, there won't be much nuclear contamination.

```
sbatch cut-trim-filter_args.sh \
  --inputDir /data/gencore/analysis_projects/8710018_Sunidhi/fastq \
  --cutadaptDir /data/gencore/analysis_projects/8710018_Sunidhi/cut-fastq \
  --adapters /data/gencore/databases/trimmomatic/all.fa
```

Next, run the fastqc and multiqc quality control on the cut fastq files.

```
module load fastqc-0.12.1-gcc-11.2.0
cd /data/gencore/analysis_projects/8710018_Sunidhi/cut-fastq
fastqc -t 2 *

mkdir -p /data/gencore/analysis_projects/8710018_Sunidhi/cut-qc
cd /data/gencore/analysis_projects/8710018_Sunidhi/cut-qc
mv /data/gencore/analysis_projects/8710018_Sunidhi/cut-fastq/*fastqc* ./

module load mamba/latest
source activate /data/biocode/programs/mamba-envs/multiqc.v1.20/
multiqc *
```

There is not much difference between the raw and processed fastq files, because the initial quality was quite good.

Step 2: getOrganelle Assemble

A lot of the parameters for this script are automatically detected, while others will need to be optimized; I chose the parameters for the first attempt somewhat randomly.

```
python /data/biocode/programs/mamba-envs/getOrganelle-env/bin/get_organelle_from_reads.py \
-1 SID_R1.fastq.gz -2 SID_R2.fastq.gz -o SID_chloroplast_output \
-R 15 -d 21,45,65,85,105 -F emblant_pt
```

I ran multiple settings (included inside the sbatch script `getOrganelle.sh`) but this setting provided by far the best graph. The graph selected and cleaned by getOrganelle's algorithms contained the expected large loop / short loop structure with a connecting repeated region, although the repeated region was shorter than typical. There were only a few unresolved loops.

Step 3: Annotation

Annotate with PGA

PGA performs reference-based annotation of circular plastid genomes.

I downloaded GenBank files for all Cactaceae chloroplast complete genomes available on NCBI, a total of 106 sequences, to use as the references.

```
perl /data/biocode/programs/PGA/PGA.pl \
-r /data/gencode/analysis_projects/8710018_Sunidhi/cactaceae-chloroplasts \
-t /data/gencode/analysis_projects/8710018_Sunidhi/getOrganelle_output/pga
```

This output looks great - well-annotated GenBank formatted file. I created a bed file specifically for the Photosystem I genes present in the genome (psaA, psaB, psaC, psal, and psaJ) to extract their sequences from the fasta file.

BED files are considered to be 0-based (the first nucleotide in the chromosome is 0) while GenBank files are 1-based (the first nucleotide in the chromosome is 1). So, I used the guidance on this page: <https://www.biostars.org/p/84686/> to convert accurately.

```
grep -B 1 'psa' emblant_pt.K105.complete.graph1.1.path_sequence_renamed.gb
gene      complement(45762..48014)
          /gene="psaA"
CDS       complement(45762..48014)
          /gene="psaA"
--
gene      complement(43532..45736)
          /gene="psaB"
CDS       complement(43532..45736)
          /gene="psaB"
--
gene      118576..118821
          /gene="psaC"
CDS       118576..118821
          /gene="psaC"
--
gene      68883..68993
          /gene="psaI"
CDS       68883..68993
          /gene="psaI"
--
gene      complement(78780..78908)
          /gene="psaJ"
CDS       complement(78780..78908)
          /gene="psaJ"
```

```
touch psa.bed
echo -e "chr1\t45761\t48014\tpsA\t1\t-" >> psa.bed
echo -e "chr1\t43531\t45736\tpsB\t1\t-" >> psa.bed
echo -e "chr1\t118575\t118821\tpsC\t1\t+" >> psa.bed
```

```
echo -e "chr1\t68882\t68993\tpsaI\t1\t+" >> psa.bed
echo -e "chr1\t78779\t78908\tpsaJ\t1\t-" >> psa.bed

module load bedtools2-2.31.0-gw

bedtools getfasta -fi ../embplant_pt.K105.complete.graph1.1.path_sequence_renamed.fasta \
                -bed psa.bed -fo psa.fa

gcsplit psa.fa "/>/" "{4}" # then rename the individual files by their gene name
```

Visualize Annotations with Chloroplot

The code for this is in the file annotation-visualization.R. I used the R package `circlize` to visualize the circular genome.

Files Returned

Raw Fastq/QC

Cut/Trimmed Fastq/QC

getOrganelle Output

- * Optimized assembly files:
 - * graph1.1.path renamed fasta/fai
 - * graph1.1.path renamed gff3
 - * graph1.selected gfa assembly graph
 - * screen shot of selected assembly graph
- * Complete assembly output

PGA Output (Annotation GenBank file)

- * Cactaceae reference genomes
 - * GenBank annotation file
 - * Annotation figure
 - * Photosystem I fasta files

Scripts

- * cut-trim-filter
- * annotation-visualization.R
- * pga.sh
- * getOrganelle.sh